

# ORGNZ-01: Introduction to Organizing Data in Grail

**Series:** ORGNZ — Organize Data: Buckets, Segments, Security | **Notebook:** 1 of 10  
| **Created:** January 2026 | **Last Updated:** 09/24/2026

## Overview

Dynatrace Grail organizes data in **buckets**, **tables**, and **views** to ensure efficient storage, flexible access, and scalable querying. Understanding how to organize your data is fundamental to achieving optimal query performance, compliance requirements, and access control.

## Prerequisites

| Requirement           | Details                                                            |
|-----------------------|--------------------------------------------------------------------|
| Dynatrace Environment | SaaS environment with Grail enabled                                |
| Permissions           | <code>storage:buckets:read</code> , <code>storage:logs:read</code> |
| Knowledge             | Basic Dynatrace navigation                                         |
| Data                  | At least 1 hour of log data                                        |

## Table of Contents

- 1. [Why Organize Data?](#)
- 2. [The Grail Data Model](#)
- 3. [Three Pillars of Data Organization](#)

4. [When to Use Each Mechanism](#)
  5. [Permission Levels in Grail](#)
  6. [Exploring Your Data Organization](#)
  7. [Organization Decision Framework](#)
- 

## Learning Objectives

---

By the end of this notebook, you will:

- Understand why data organization matters in Grail
- Know the three pillars of data organization: buckets, segments, and security
- Recognize when to use each organization mechanism
- Understand the Grail data model structure

## Why Organize Data?

---

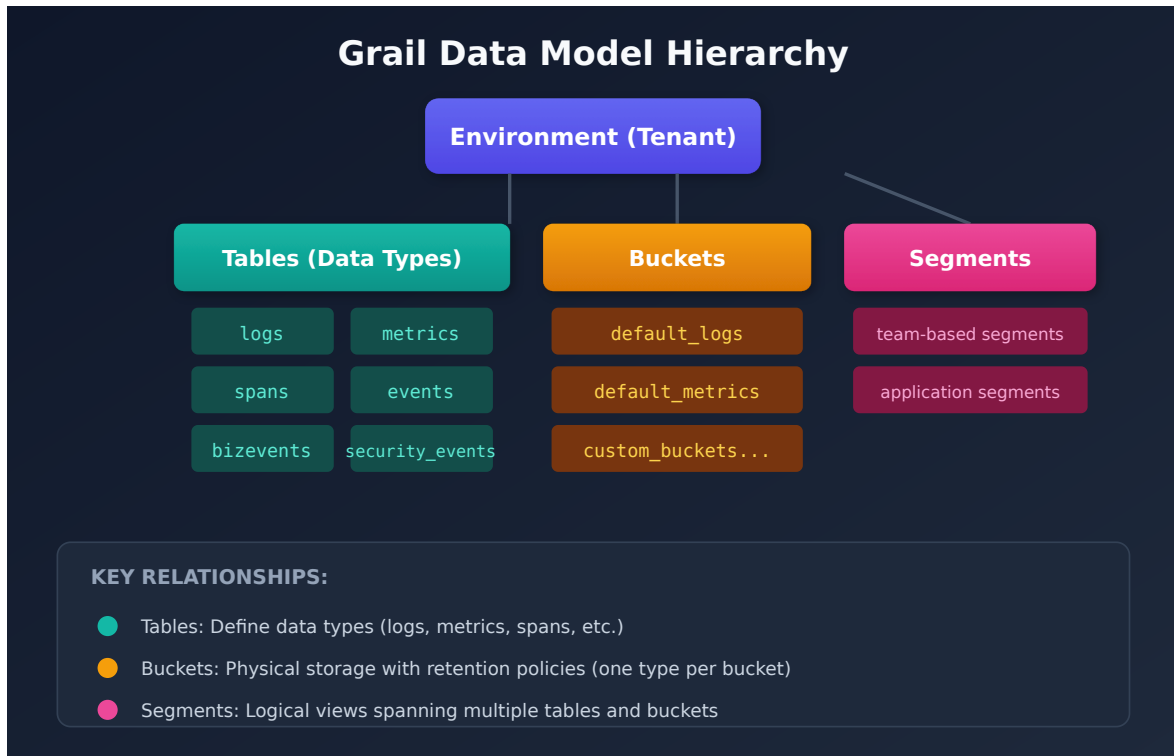
Effective data organization in Grail enables:

| Benefit             | Description                                               |
|---------------------|-----------------------------------------------------------|
| Query Performance   | Reduce query execution time by limiting data scope        |
| Cost Control        | Transparent GiB/day pricing with bucket-level attribution |
| Access Control      | Fine-grained permissions from bucket to field level       |
| Compliance          | Meet retention and data residency requirements            |
| Operational Clarity | Teams see only relevant data for faster troubleshooting   |

## The Grail Data Model

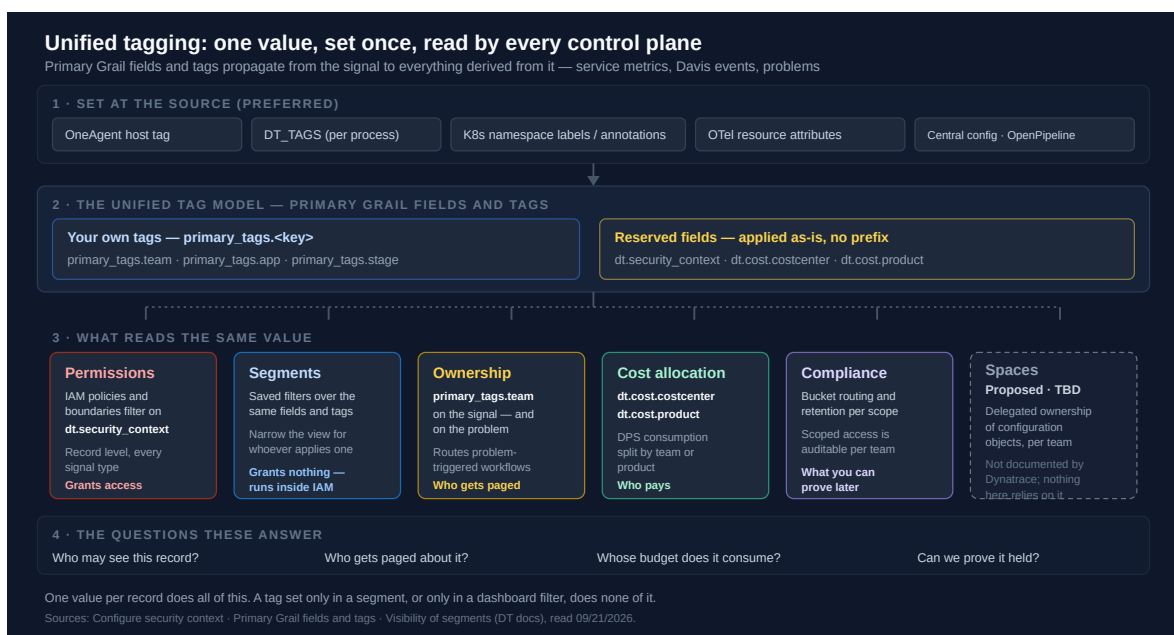
---

Grail organizes data hierarchically with tables (data types), buckets (storage containers), and segments (logical views):



## Three Pillars of Data Organization

All three pillars read metadata that was set once, upstream of them. This is the layered view — where tagging happens, and what consumes it:



Dynatrace states the ordering plainly: *"At-source enrichment is always preferred over OpenPipeline-based enrichment for the security context."* The value then travels, because

"Dynatrace enriches all derived signals (service metrics, Davis events, and problems) with the same tags." Buckets, segments and security context are three consumers of that one value; ownership routing and cost allocation are two more. **FAQ-02** covers the tagging side in full.

**Spaces — proposed, TBD.** The gap is real today: `dt.security_context` delegates which *records* a team may read, but nothing delegates which *configuration objects* a team may own — changing an alert, an SLO or a maintenance window still goes through whoever holds the settings permissions, which is how a central team becomes a queue. A **Spaces** construct — delegated ownership of configuration objects, per team — is the proposed answer, and it is drawn here to mark where it would sit. It is a concept, not a capability: Dynatrace publishes no documentation for it as of 09/21/2026, no timing is announced, and nothing in this entry depends on it. Design against what the other columns give you today.

Sources: [Configure security context \(DT docs\)](#), [Primary Grail fields and tags \(DT docs\)](#).

## 1. Buckets (Physical Organization)

Buckets are logical storage containers where records are stored:

| Aspect           | Description                                      |
|------------------|--------------------------------------------------|
| <b>Purpose</b>   | Physical data partitioning and retention control |
| <b>Scope</b>     | One data type per bucket (logs, metrics, etc.)   |
| <b>Retention</b> | 1 day to 10 years per bucket                     |
| <b>Access</b>    | IAM policies can grant bucket-level access       |
| <b>Cost</b>      | Enable precise cost attribution by team/LOB      |

## 2. Segments (Logical Organization)

Segments provide dynamic, cross-signal data views:

| Aspect         | Description                                     |
|----------------|-------------------------------------------------|
| <b>Purpose</b> | Real-time filtering without physical separation |

| Aspect      | Description                                     |
|-------------|-------------------------------------------------|
| Scope       | Spans multiple data types and buckets           |
| Flexibility | Rule-based, can use variables and relationships |
| Access      | Governed by existing access controls            |
| Cost        | No direct storage cost impact                   |

### 3. Security Context (Access Control)

Security context enables fine-grained permissions:

| Aspect      | Description                                           |
|-------------|-------------------------------------------------------|
| Purpose     | Record-level access control                           |
| Mechanism   | <code>dt.security_context</code> attribute on records |
| Flexibility | Custom values, hierarchical encoding                  |
| Enforcement | IAM policies filter at query time                     |

## When to Use Each Mechanism

| Need                        | Solution                | Rationale                              |
|-----------------------------|-------------------------|----------------------------------------|
| Different retention periods | <b>Buckets</b>          | Retention is set per bucket            |
| Cost attribution by team    | <b>Buckets</b>          | Direct GiB/day billing visibility      |
| Broad team isolation        | <b>Buckets</b>          | Simple bucket-level IAM policies       |
| Dynamic data views          | <b>Segments</b>         | Filter across data types at query time |
| Application-centric views   | <b>Segments</b>         | Group related entities and signals     |
| Record-level access control | <b>Security Context</b> | Fine-grained ABAC                      |
| Multi-team shared buckets   | <b>Security Context</b> | Filter records within same bucket      |

# Permission Levels in Grail

Grail supports permissions at multiple levels:

| Level  | Granularity                     | Example Use Case                           |
|--------|---------------------------------|--------------------------------------------|
| Bucket | All records in a bucket         | Team owns entire bucket                    |
| Table  | All records of a data type      | Access to all logs                         |
| Record | Individual records by attribute | By host group, namespace, security context |
| Field  | Specific fields on records      | Hide sensitive fields (fieldsets)          |

**Important:** Without permissions, users cannot query data from Grail. Permissions must be explicitly granted.

# Exploring Your Data Organization

## DQL: Bucket Discovery

Use the following query to explore your Grail bucket configuration:

```
// List all buckets – foundational discovery query for your Grail environment
fetch dt.system.buckets
| fields name, display_name, dt.system.table, retention_days,
estimated_uncompressed_bytes
| sort dt.system.table asc, name asc
```

# Organization Decision Framework

Use this framework to decide how to organize your data:



## Series Overview

This series covers data organization in depth:

| Notebook | Topic                | Focus                                                     |
|----------|----------------------|-----------------------------------------------------------|
| ORGNZ-01 | Introduction         | Why and how to organize data                              |
| ORGNZ-02 | Grail Buckets        | Bucket fundamentals and limits                            |
| ORGNZ-03 | Bucket Strategy      | Naming, retention, design patterns                        |
| ORGNZ-04 | Permissions Overview | Permission levels and policy management                   |
| ORGNZ-05 | Bucket-Level Access  | IAM policies for bucket access                            |
| ORGNZ-06 | Security Context     | Fine-grained access with <code>dt.security_context</code> |

| Notebook | Topic                        | Focus                                                 |
|----------|------------------------------|-------------------------------------------------------|
| ORGNZ-07 | Advanced Permissions         | Record-level and field-based access                   |
| ORGNZ-08 | Grail Segments               | Dynamic data organization                             |
| ORGNZ-09 | Enterprise Patterns          | Combined approaches and best practices                |
| ORGNZ-10 | Advanced Segment Definitions | Filter syntax, enrichment, variables, troubleshooting |

## Next Steps

---

Continue with the ORGNZ series:

- **ORGNZ-02**: Understanding Grail Buckets

## References

---

- [Organize data](#)
  - [Permissions in Grail](#)
  - [Grail data model](#)
- 

*This notebook was AI-generated from Dynatrace documentation and enterprise best practices. It is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*